Universidade Federal Rural de Pernambuco
Departamento de Estatística e Informática
Bacharelado em Sistemas de Informação

Otimização de rotas para drones: Abordagem via força bruta para o problema do FlyFood

BRUNO FELLYPE DE SANTANA VIEIRA
ISAQUE LUCAS PEDROSA VELOZO
LUCAS MENEZES DOS SANTOS
RENATO HENRIQUE BARROS

Recife
Abril de 2026

Resumo

Com a estimativa de desenvolvimento urbano exponencial e o aumento significativo da demanda por entregas rápidas, as soluções tradicionais de empresas de *delivery* encontram dificuldades em realizar as entregas em tempo hábil. Fatores como a distância entre as entregas, o trânsito excessivo, as dificuldades de locomoção e o custo de mão de obra impactam diretamente na logística, tornando a ideia de entrega rápida muitas vezes contraditória.

Diante desse cenário, o projeto FlyFood propõe uma metodologia de entregas nova: uso de drones autônomos para otimizar o tempo e o custo das entregas no ambiente urbano. O projeto se concretiza a partir do desenvolvimento de um algoritmo capaz de calcular a rota mais eficiente para os drones realizarem a entrega, minimizando o tempo de locomoção total respeitando os limites de movimentação ortogonal do drone (vertical e horizontal).

Para isso, o algoritmo recebe uma matriz de entrada, que representa a cidade, com posições fixas para o ponto de origem R e para os pontos de entrega $\{A, B, C, \ldots\}$. O algoritmo, através do uso de força bruta, é capaz de calcular através da *Distância de Manhattan* as distâncias entre os pontos de entrega, calcula todas as possíveis rotas e seleciona a que possui a menor distância e retorna ela para que o drone possa efetuá-la. A solução desse problema é uma aplicação direta do clássico Problema do Caixeiro Viajante *(Travelling Salesman Problem - TSP)* onde o objetivo é encontrar o ciclo hamiltoniano de menor peso dentro do grafo.

Palavras-chave: Algoritmo por Força bruta, otimização de rota, problema do caixeiro viajante.

Abstract

With the projected exponential growth of urban development and the significant increase in demand for fast deliveries, traditional delivery solutions are finding it difficult to make deliveries on time. Factors such as the distance between deliveries, excessive traffic, difficulties in transportation, and labor costs directly impact logistics, making the idea of fast delivery often contradictory.

In this scenario, the FlyFood project proposes a new delivery methodology: the use of autonomous drones to optimize the time and cost of deliveries in the urban environment. The project is based on the development of an algorithm capable of calculating the most efficient route for drones to make deliveries, minimizing total travel time while respecting the drone's orthogonal movement limits (vertical and horizontal).

To do this, the algorithm receives an input matrix, which represents the city, with fixed positions for the origin point R and for the delivery points $\{A, B, C, \ldots\}$. The algorithm, through the use of brute force, is able to calculate, using the *Manhattan Distance*, the distances between the delivery points, calculates all possible routes, and selects the one with the shortest distance and returns it so that the drone can perform it. The solution to this problem is a direct application of the classic Traveling Salesman Problem *(Travelling Salesman Problem - TSP)* where the objective is to find the Hamiltonian cycle with the lowest weight within the graph.

Keywords: Brute force algorithm, route optimization, traveling salesman problem.

1 Introdução

1.1 Apresentação e Motivação

O crescimento desordenado das cidades, juntamente com a alta demanda pelo serviço de entrega tem causado congestionamentos logísticos cada vez maiores nas cidades. Veículos terrestres enfrentam constantes dificuldades de locomoção devido aos congestionamentos no trânsito, além de contribuirem na emissão de poluentes ambientais por serem movidos à combustíveis fósseis.

A viabilidade do uso de veículos aéreos não tripulados - ou drones - para mitigar esse conjunto de problemáticas é explorada por Hokama et al. (2024), que destaca o uso potencial de drones para superar barreiras geográficas terrenas. Nesse cenário, surge o **FlyFood**, com a proposta de utilizar drones autônomos elétricos para otimizar entregas urbanas. Inicialmente, o FlyFood já soluciona dois dos principais problemas de outros serviços de entrega: Otimização de entregas em diversos pontos da cidade, visto que no ar o drone não encontrará congestionamento tampouco dificuldades de locomoção e redução da dependência de combustíveis fósseis, visto que serão movidos à bateria. Os drones nesse cenário se mostram como uma alternativa significativa para serviços de entrega tanto na otimização do serviço, quanto na economia do custo operacional e na redução do impacto ambiental.

Contudo, a implementação dos drones vem com uma limitação física - a autonomia das baterias. Por isso, torna-se imperativo que os drones percorram a menor rota possível para que seu tempo de voo seja suficiente e alcançar o seu objetivo. Para isso, o uso de algoritmos eficientes para encontrar essa rota se torna o ponto alto do desenvolvimento do FlyFood. Este problema de otimização de rota se baseia no clássico problema matemático do Caixeiro Viajante *(TSP)*, como aponta Cormen et al. (2022), o TSP é um desafio clássico de otimização combinatória onde o custo computacional cresce de modo fatorial, sendo classificado por Applegate et al. (2011) como um problema **NP-Difícil**. Isso implica que não se conhece um algoritmo de tempo polinomial capaz de resolvê-lo para qualquer tamanho de entrada. Diferente das heurísticas, que podem encontrar a menor rota ou uma rota de valor próximo, o algoritmo por força bruta encontra sempre com exatidão a menor rota considerando a distância ortogonal entre os pontos através da Distância de Manhattan, porém sua complexidade aumenta conforme os pontos de entrega aumentam, visto que se trata de um problema de complexidade $O(n!)$, o que torna essa solução inviável no mundo real para uma quantidade muito grande de pontos de entrega.

Esse artigo tem a intenção de descrever o processo de implementação de um algoritmo de otimização combinatória por força bruta para o FlyFood, detalhando a lógica de desenvolvimento, análise de complexidade e análise de resultados obtidos nos testes de rota.

1.2 Formulação do problema

De maneira geral, o problema consiste em receber uma matriz bidimensional que representa a malha da cidade, realizando a identificação precisa do ponto de origem e retorno denotado como R e dos diversos pontos de demanda distribuídos na malha. O desafio reside em determinar a sequência ideal de visitação que minimize a distância total percorrida, configurando-se como uma aplicação direta do clássico Problema do Caixeiro Viajante (Traveling Salesperson Problem - TSP) Hoffman et al. (2013).

Diferente de modelos que utilizam a distância euclidiana, o sistema FlyFood adota a **Distância de Manhattan**, uma vez que o deslocamento do drone é restrito aos eixos

horizontal e vertical da matriz. Essa métrica, também chamada de Geometria do Táxi, é amplamente fundamentada por Krause (1973). Matematicamente, a distância entre dois pontos $P_1(x_1, y_1)$ e $P_2(x_2, y_2)$ é dada por:

$$d = |x_1 - x_2| + |y_1 - y_2|$$

Desse modo, a complexidade desse problema está relacionada ao crescimento do número de busca que cresce de maneira fatorial, graças ao aumento no número de entregas a serem feitas, cabendo ao software ser capaz de identificar a rota de menor custo distancial possível.

1.3 Objetivos

1.3.1 Objetivo geral

Desenvolver um algoritmo capaz de determinar a rota de menor custo para que um drone realize entregas em todos os pontos definidos em uma matriz de entrada, garantindo que o percurso se inicie e se encerre no ponto de origem, denominado R.

1.3.2 Objetivos específicos

Os objetivos específicos deste trabalho incluem modelar o problema de entregas como um sistema baseado em coordenadas em uma matriz bidimensional; implementar o cálculo de distâncias entre pontos por meio da métrica de Manhattan; identificar automaticamente o ponto de origem R e os pontos de entrega na matriz; gerar as possíveis sequências de visita aos pontos de entrega; calcular o custo total de cada rota, considerando o percurso de ida e retorno ao ponto de origem; determinar a sequência que minimize a distância total percorrida; e, por fim, avaliar as limitações da abordagem adotada, bem como possíveis melhorias.

1.4 Organização do trabalho

O presente trabalho está organizado da seguinte maneira:

1. **Introdução:** Apresenta o contexto do problema de entregas por drones, qual problema que estará sendo tratado e os objetivos traçados para solucioná-lo.

2. **Referencial Teórico:** Apresenta os principais conceitos, métodos e técnicas referenciados pelo trabalho.

3. **Trabalhos Relacionados:** Cita outros projetos, estudos, pesquisas e soluções já existentes na literatura, situando o trabalho em relação a outras abordagens.

4. **Metodologia:** Explica como o problema foi modelado e detalha a abordagem e os procedimentos adotados para desenvolver o algoritmo.

5. **Experimentos:** Descreve como os testes foram realizados, incluindo os cenários, entradas utilizadas e critérios de avaliação.

6. **Resultados:** Apresenta e analisa os dados obtidos nos experimentos, evidenciando o desempenho do algoritmo.

7. **Conclusão:** Resume os principais resultados, discute limitações e aponta possíveis melhorias.

2 Referencial Teórico

O projeto FlyFood é uma derivação direta do *Traveling Salesman Problem* - o Problema do Caixeiro Viajante com uma metodologia atrelada à técnicas matemáticas e anĺises computacionais como: Análise de Complexidade de Algortimo, Algoritmo por Força Bruta, métrica da *Distância de Manhattan*, representações em grafos e matrizes e mecanismos de poda.

2.1 O Problema do Caixeiro Viajante (TSP)

O problema de otimização de rotas abordado no projeto FlyFood encontra seus fundamentos teóricos no clássico Problema do Caixeiro Viajante (Traveling Salesperson Problem - TSP). Segundo Cormen et al. (2022), o objetivo é encontrar o ciclo hamiltoniano de menor peso, ou seja, determinar o menor caminho fechado em um grafo. Applegate et al. (2011) reforçam que por ser um problema da classe **NP-Difícil**, o número de rotas possíveis a serem analisadas cresce de modo fatorial $(n-1)!$. Em outras palavras, o algoritmo deve determinar o caminho fechado mais curto que visite todos os pontos de demanda exatamente uma única vez e, em seguida, retorne rigorosamente ao seu ponto de origem. O que impõe limitações significativas ao utilizar em larga escala.

2.2 Algoritmo de Força Bruta e Complexidade Computacional

Na literatura da ciência da computação, existem diversos métodos heurísticos que entregam soluções aproximadas em tempos reduzidos. Contudo, a abordagem adotada neste trabalho baseia-se em um algoritmo de busca exata, conhecido como Força Bruta. A estratégia da força bruta, ou busca exaustiva, consiste em examinar todas as soluções candidatas até encontrar a ideal.

Conforme Sipser (1996), embora essa abordagem garanta a solução ótima sempre, sua complexidade de tempo é proibitiva para valores altos de n, resultando em uma **explosão combinatória**. Isso significa que o número de rotas possíveis aumenta exponencialmente a cada novo ponto adicionado, limitando sua aplicação prática. Por exemplo, para um cenário com 10 cidades, o sistema exige a avaliação de mais de **3,6 milhões de rotas**.

2.3 Métrica de Distância de Manhattan

Diferente de modelos que utilizam a distância euclidiana, considerando o percurso em linha reta, o modelo do FlyFood opera em um ambiente urbano estruturado utilizando da **métrica de Manhattan**. De acordo com Krause (1973), essa métrica é ideal para modelos representados por matrizes ou grades urbanas onde o movimento é restrito aos eixos ortogonais como o caso do FlyFood. Consequentemente, para modelar matematicamente essa restrição, utiliza-se a **Distância de Manhattan**. A distância entre dois pontos é calculada por meio da soma das diferenças absolutas de suas coordenadas: $d = |x_1 - x_2| + |y_1 - y_2|$.

2.4 Representação em Grafos e Matrizes Esparsas

No projeto FlyFood, a malha urbana é modelada como uma **matriz esparsa bidimensional**. Como a matriz da cidade possui muitos espaços vazios - onde não ocorrem entregas - ela pode ser classificada como uma matriz esparsa. Segundo Goodrich et al.

(2014), o armazenamento eficiente dessas estruturas é fundamental para poupar memória, processando apenas os elementos que possuem informação realmente útil para o algoritmo. A malha urbana do FlyFood permite o problema seja tratado também como um conjunto de vértices e arestas construindo um **grafo**, que conforme Cormen et al. (2022) acaba por ser uma estrutura ideal para modelagem de conexões de rede, onde os pontos de entrega são os vértices e os custos de deslocamento são as arestas. Essa abstração teórica é fundamental, pois é o que permite transformar um mapa físico e visual em uma estrutura de dados matemática processável por algoritmos.

2.5 Árvores de Busca e a Teoria da Poda *(Pruning)*

Embora a abordagem de Força Bruta avalie empiricamente todas as combinações, a teoria da ciência da computação estabelece mecanismos de otimização matemática como árvores de busca, que são definidas por Russell (2010) como uma representação de todas as sequências possíveis de ações, partindo da raiz (ponto R) até as folhas (sequência de entregas). Com isso, para otimizar essa estrutura, a Poda *(Pruning)* se torna um dos conceitos chaves. De acordo com Knuth (1997), a poda permite descartar galhos que comprovadamente não leverão a uma solução melhor que uma já encontrada, reduzindo o esforço computacional sem perder a garantia da solução ótima. Em uma árvore de permutações, cada ramificação representa um caminho em potencial. A teoria por trás do pruning dita que, ao definir um limite superior *(upper bound)* — que no algoritmo é representado pela distância da melhor rota já encontrada —, o sistema passa a **avaliar o custo parcial de um caminho em tempo real**. Se o custo acumulado em um galho intermediário já **ultrapassa o limite superior**, a teoria prova matematicamente que nenhum caminho derivado daquele galho poderá ser a solução ótima.

3 Trabalhos Relacionados

O problema de roteamento e logística urbana possui uma vasta literatura, tendo como alicerce fundamental o **Problema do Caixeiro Viajante (TSP)**. Conforme definido pelo trabalho de Hoffman et al. (2013) , o *TSP* é um desafio amplamente estudado na otimização combinatória , onde o objetivo é encontrar o ciclo hamiltoniano de menor peso dentro de um grafo. O projeto FlyFood utiliza essa base teórica clássica de Hoffman et al. (2013) para aplicá-la em um cenário tecnológico moderno: o uso de drones autônomos para otimizar o tempo e o custo das entregas.

Enquanto os estudos seminais de Hoffman et al. (2013) focam na formulação matemática abstrata do caixeiro viajante, trabalhos como os de Dorling et al. (2016) buscam aplicar esses conceitos para contornar as limitações físicas de veículos terrestres como o trânsito excessivo e a dificuldade de locomoção urbana. Esses trabalhos evidenciam que a transição de veículos movidos a combustíveis fósseis para Veículos Aéreos Não Tripulados *(VANTs)* reduz significativamente o tempo de entrega, mas introduz um novo obstáculo: a limitação da autonomia das baterias.

Dessa forma, o FlyFood complementa a literatura existente ao focar na exigência de um algoritmo de Força Bruta para garantir a execução da rota absolutamente mais curta, viabilizando a operação de entrega dentro do tempo de voo disponível, mitigando o novo obstáculo encontrado.

4 Metodologia

A metodologia utilizada neste trabalho consistiu em um conjunto de técnicas e ferramentas que configuram a entrada que o programa recebe, o processamento que é feito e a formatação da saída. Utilizando a linguagem de programação **Python** como ferramenta de programação, o editor de texto **VSCode** como ambiente de desenvolvimento e o repositório remoto **GitHub** para armazenar o código-fonte e os arquivos de testes acessível pelo seguinte link: github.com/lucasmenezes255/flyfood.

4.1 Leitura e Estruturação Espacial dos Dados

O processo inicia-se abstraindo a área de operação do drone como uma matriz esparsa bidimensional, varrendo um conjunto de arquivos de texto de entrada, que representam as malhas urbanas com diferentes números de pontos: 5, 8, 10, 11, 12, 13 e 14. Para otimizar o acesso à memória durante a execução, o algoritmo descarta a necessidade de percorrer a matriz repetidas vezes. Em vez disso, ao localizar uma coordenada vália (o ponto de origem R ou os pontos de entrega), o sistema armazena seu rótulo e sua respectiva **tupla de posição da matriz (i, j)** em uma estrutura de dicionário **(Tabela Hash)**. Essa arquitetura garante que a consulta de localização de qualquer cidade ocorra em tempo constante $O(1)$, maximizando a performance da leitura.

4.2 Pré-processamento de Distâncias

Antes de iniciar a geração de rotas, o algoritmo executa uma etapa crucial de otimização matemática. Utilizando o método ***combinations*** da biblioteca python ***itertools***, o programa mapeia todos os pares possíveis de cidades presentes na malha e calcula, de forma antecipada, a **Distância de Manhattan** entre elas. Esses valores são armazenados em um **dicionário de distâncias**. Essa estratégia de memorização **evita que o computador recalcule a distância entre os mesmos dois pontos** (por exemplo, de A para B) milhões de vezes durante a fase de análise de rotas, economizando um poder de processamento significativo.

4.3 Busca Exaustiva e Geração de Rotas (Força Bruta)

Com o ambiente e as distâncias mapeadas, o núcleo do algoritmo entra em ação para garantir a descoberta do ótimo global. Através do método ***permutations*** - outro método da lib ***itertools***, o sistema gera de **forma exaustiva** todas as ordens possíveis de visitação aos pontos de entrega, caracterizando a complexidade fatorial $O(N!)$ do problema. Cada permutação gerada representa uma sequência candidata. O programa itera sobre essa lista, somando os valores de distância (previamente calculados na etapa anterior) de ponto a ponto, garantindo que o custo de retorno ao ponto de origem R seja rigorosamente adicionado ao final de cada trajeto.

4.4 Otimização por Poda de Árvore de Busca *(Pruning)*

Para mitigar a *explosão combinatória* inerente ao cálculo de permutações, implementou-se uma técnica de poda *(pruning)* na função de avaliação de rotas. O sistema inicializa registrando o custo da primeira rota completa encontrada como o seu *limite superior (upper bound)*. A partir daí, durante a avaliação de novas ramificações, o algoritmo **verifica o**

custo parcial em tempo real. Se, no meio de um trajeto, a soma das distâncias parciais igualar ou ultrapassar o limite superior vigente, a função aborta imediatamente a leitura daquela rota e pula para a próxima rota da iteração. Essa interrupção precoce impede que o processador realize somas inúteis em caminhos que, matematicamente, já não possuem chance de configurar a solução ótima.

5 Experimentos

Para validar a corretude e avaliar o desempenho do algoritmo de Força Bruta, foi desenvolvida uma suíte de testes automatizados. O experimento consistiu em submeter o algoritmo a ler arquivos de textos possuindo matrizes de tamanhos variáveis, incrementando gradativamente o número de pontos de entrega (cidades) de $N = 4$ até $N = 14$.

A métrica principal de avaliação foi o **Tempo de Execução (em segundos)**, isolando estritamente a função de cálculo das permutações e descartando o tempo de leitura de arquivos e impressões na tela. Devido à natureza exponencial do problema, foi definida uma tolerância de execução; testes que ultrapassassem 24 horas para uma única matriz marcariam o limite prático da abordagem.

6 Resultados

Os dados obtidos confirmam a complexidade teórica do algoritmo de Força Bruta para o Problema do Caixeiro Viajante, cujo crescimento se dá em ordem fatorial $O(N!)$. A Tabela 1 apresenta o tempo médio de execução em relação ao número de pontos de entrega.

Quantidade de Cidades	**Rotas Possíveis ($N!$)**	**Tempo de Execução (s)**
4	24	0,00005
8	40.320	0,04494
10	3.628.800	2,89593
11	39.916.800	32,79216
12	479.001.600	442,89283 ($\approx$ 7 min)
13	6.227.020.800	7.618,07 ($\approx$ 2 horas)
14	87.178.291.200	90.456,31 ($\approx$ 25 horas)

Tabela 1: Relação entre o número de cidades e o tempo de execução.

Observa-se que para quantidades pequenas de entregas ($N \leq 10$), o algoritmo encontra a rota ótima em poucos segundos. No entanto, o tempo de processamento sofre uma "explosão combinatória"a partir de 11 cidades. O salto de 13 para 14 cidades ilustra perfeitamente a limitação da Força Bruta: a adição de apenas um único ponto de entrega aumentou o tempo de processamento de aproximadamente **2 horas para mais de 25 horas**.

Figura 1: Desempenho do algoritmo: Escala Linear.

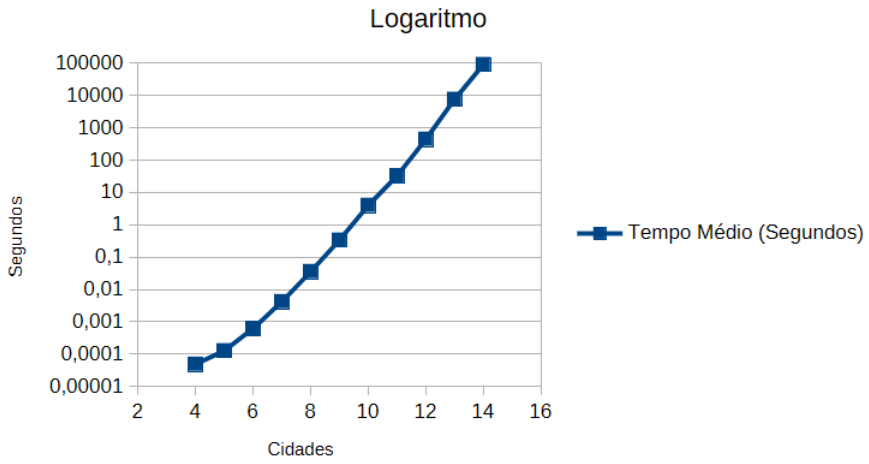

Figura 2: Desempenho do algoritmo: Escala Logarítmica.

Os gráficos das Figuras 1 e 2 ilustram visualmente a inviabilidade de escalar a solução. Enquanto o gráfico linear evidencia o momento exato da "explosão combinatória", o gráfico logarítmico comprova que o algoritmo sofre um aumento de complexidade constante e acentuado a cada nova cidade adicionada. Apesar da abordagem garantir matematicamente a rota de menor custo (ótimo global), o custo computacional torna-se proibitivo para a logística diária de um drone do *FlyFood* operando em larga escala.

7 Conclusão

O presente trabalho permitiu o desenvolvimento de um algoritmo capaz de determinar a rota de menor custo para entregas realizadas por drones, garantindo o início e o retorno ao ponto de origem. A abordagem adotada, baseada em força bruta, mostrou-se eficiente para um número reduzido de pontos de entrega, assegurando a obtenção da solução ótima.

Entretanto, os resultados obtidos evidenciaram a limitação prática do método devido à sua complexidade fatorial O(N!). Observou-se que, a partir de 11 pontos de entrega, o tempo de execução cresce de forma acentuada, tornando-se inviável para aplicações em larga escala. O aumento de aproximadamente 2 horas para mais de 25 horas ao passar de 13 para 14 pontos ilustra claramente a explosão combinatória do problema.

Dessa forma, conclui-se que, embora a abordagem garanta a melhor rota possível, seu custo computacional restringe sua aplicação em cenários reais. Assim, para sistemas logísticos como o FlyFood, torna-se recomendável a utilização de métodos mais eficientes, como heurísticas ou algoritmos aproximativos, conforme sugerido por Kuroswiski et al. (2021) onde o uso de métodos aproximativos permite encontrar soluções satisfatórias com um custo computacional drasticamente menor, adequando-se às limitações de tempo e processamento real dos drones.

Referências Bibliográficas

Applegate, D. L., Bixby, R. E., Chvátal, V., and Cook, W. J. (2011). The traveling salesman problem: a computational study. In *The traveling salesman problem*. Princeton university press.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). *Introduction to algorithms*. MIT press.

Dorling, K., Heinrichs, J., Messier, G. G., and Magierowski, S. (2016). Vehicle routing problems for drone delivery. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 47(1):70–85.

Goodrich, M. T., Tamassia, R., and Goldwasser, M. H. (2014). *Data structures and algorithms in Java*. John wiley & sons.

Hoffman, K. L., Padberg, M., Rinaldi, G., et al. (2013). Traveling salesman problem. *Encyclopedia of operations research and management science*, 1:1573–1578.

Hokama, P. H., Lintzmayer, C. N., and San Felice, M. C. (2024). Sobre o problema do caixeiro viajante com drone. In *Encontro de Teoria da Computação (ETC)*, pages 30–37. SBC.

Knuth, D. E. (1997). *The Art of Computer Programming: Fundamental Algorithms, Volume 1*. Addison-Wesley Professional.

Krause, E. F. (1973). Taxicab geometry. *Mathematics Teacher*, 66(8):695–706.

Kuroswiski, A. R., Pires, H. B., Passaro, A., Frutuoso, L. N., and Guimarães, E. L. F. S. (2021). Abordagens exata e heurística na otimização do problema do caixeiro viajante com drone.

Russell, P. N. (2010). Artificial intelligence: a modern approach by stuart. *Russell and Peter Norvig contributing writers, Ernest Davis...[et al.]*, page 22.

Sipser, M. (1996). Introduction to the theory of computation. *ACM Sigact News*, 27(1):27–29.